אוניברסיטת בן-גוריון
מדור בחינות

מספר נבחן:_____

תאריך הבחינה: 31.3.2024
שם המורה: ד"ר מני אדלר
יאיר ועקנין
שם הקורס: תכנות מערכות
מספר הקורס: 202-1-2031
שנה: תשפ"ד
סמסטר: א
מועד: א
משך הבחינה: שעתיים
חומר עזר: אין

רשמו תשובותיכם בגיליון התשובות בלבד, תשובות מחוץ לגיליון לא יבדקו.
ניתן לענות 'לא יודע' על סעיף שלם (בלבד) ולקבל 20% מהניקוד עבורו

בהצלחה!

שאלה 1: זיכרון (35 נקודות)

נתונה התוכנית הבאה ב-**Java**:

```
class BinaryTree<T extends Comparable<T>> {
    private T data;
    private BinaryTree<T> left;
    private BinaryTree<T> right;

    BinaryTree(T data) {
        this.data = data;  this.left = null ; this.right = null;
    }

    BinaryTree(T data, BinaryTree left, BinaryTree right) {
        this.data = data;  this.left = left; this.right = right;
    }
    public T getData() { return data; }

    public BinaryTree<T> getLeft() { return left; }

    public BinaryTree<T> getRight() { return right; }

    public void insert(T data) {
         if (this.data.compareTo(data) == 0)
            return;
```

```
        if (this.data.compareTo(data) > 0) {
            if (left == null)
                left = new BinaryTree<T>(data);
            else
                left.insert(data);
        } else {
            if (right == null)
                right = new BinaryTree<T>(data);
            else
                right.insert(data);
        }
    }
}
```

א. השלימו את המימוש הבסיסי של המחלקה ב-C++, המופיע בגליון התשובות

הערות:

- המתודה compareTo ב-Java מחזירה 1 אם this גדול מ-other, ו -1 אם this קטן מ-other.
- ניתן להניח כי בטיפוס T בתבנית ממומשים כל ה-operator overloadings, כמו ==operator וכדומה)

[10 נקודות]

ב. השלימו את מימוש המחלקה על פי תבנית Rule of 5

הערות:

- כדי לחסוך זמן, ניתן לממש את כל המתודות במחלקה עצמה ב-header (מבלי ליצור כפילות של הצהרות ומימוש), כפי שנעשה בסעיף א.
- ניתן להוסיף פונקציות עזר

[15 נקודות]

ג. ציירו את תמונת הזיכרון בנקודה המסומנת [10 נקודות]

```
{
    BinaryTree<int> t(1);
    t.insert(2);
    t.insert(3);

    f(BinaryTree<int>(1, new BinaryTree<int>(2), new BinaryTree<int>(3)));
}

void f(BinaryTree<int> t) {
    // @@ Draw the memory at this point
    ...
}
```

שאלה 2: מקביליות (30 נקודות)

נתון הממשק Set המייצג **קבוצה** של אובייקטים:

```
public interface Set<T> {
    boolean contains(T obj);
    void add(T obj);
}
```

א. הגדירו תכונה נשמרת (אינווריאנטה) לממשק, ותנאי התחלה וסיום למתודה add. אם נדרש, הרחיבו את הממשק לשם כך.
[10 נקודות]

נתון מימוש של המתודה add (מימוש המתודה contains אינו נדרש לשאלה):

```
class SetImpl<T> implements Set<T> {

    private volatile Link<T> head;

    SetImpl() {
        head = null;
    }

    public boolean contains(T obj) {
        ...
    }

    public void add(T obj) {
        if (head == null)
            head = new Link<T>(obj,null);
        else
            add(head,obj);
    }
```

```
    private void add(Link<T> curr, T obj) {
        if (curr.data.equals(obj))
            return;
        if (curr.next == null)
           curr.next = new Link(obj,null);
        else
           add(curr.next,obj);
    }

    private static final class Link<T> {
        private T data;
        private volatile Link<T> next;

        Link(T data, Link<T> next) {
            if (data == null)
               throw new Exception("Null Data");
            this.data = data;
            this.next = next;
        }

    }

}
```

ב. האם המחלקה **SetImpl** בטוחה בחישוב מקבילי? אם כן, נמקו בקצרה. אם לא, הראו תרחיש לא בטוח.
[5 נקודות]

ג. האם המחלקה **SetImpl** נכונה בחישוב מקבילי? אם כן, נמקו בקצרה. אם לא, הראו תרחיש לא נכון.
[5 נקודות]

ד. סנכרנו את **add** בגישת lock-free כך שהיא תהיה בטוחה ונכונה.
[10 נקודות]

שאלה 3: תקשורת ובסיסי נתונים (35 נקודות)

בנספח (בסוף השאלון), מופיע לנוחיותכם קוד הריאקטור כפי שנלמד בכיתה.

אחד הסטודנטים בקורס החליט לייעל את עבודת יחידת התקשורת בריאקטור, על ידי עדכון המתודה **handleReadWrite** במחלקה **Reactor** באופן הבא:

```
Class Reactor {

  ExecutorService communicationThreads = Executors.newFixedThreadPool(2);

  ...

 private void handleReadWrite(SelectionKey key) {
        @SuppressWarnings("unchecked")
        NonBlockingConnectionHandler<T> handler =
                    (NonBlockingConnectionHandler<T>) key.attachment();
        if (key.isReadable()) {
            communicationThreads.submit( () -> {
               Runnable task = handler.continueRead();
               if (task != null)
                   pool.submit(handler, task);
               else if (!key.isValid())
                   return;
            });
        }
        if (key.isWritable()) {
           communicationThreads.submit( () -> {
               handler.continueWrite();
           });
        }
    }

    ...
}
```

בתיעוד של Java מצוין כי:

- המחלקה SocketChannel היא Thread Safe באופן הבא:
  - בביצוע מקבילי של המתודה read שני ת'רדים לא יקראו בו זמנית מה-channel
  - בביצוע מקבילי של המתודה write שני ת'רדים לא יכתבו בו זמנית ל-channel
  - שני ת'רדים יכולים לבצע במקביל וללא חשש read ו-write
- המחלקה ByteBuffer אינה Thread Safe

א. ציינו 4 תרחישים בעייתיים **<u>שונים</u>** (כלומר 4 בעיות **<u>שונות</u>)** העשויים להתרחש בהרצת ההצעה של הסטודנט.
[20 נקודות]

ב. תקנו את הקוד של הצעת הסטודנט כך שלא תהיה בו כל בעיה. בחרו בפיתרון היעיל ביותר מבחינת מקבול המשימות (התיקון צריך לשמר את ההצעה של הסטודנט בדבר 2 ת'רדים נוספים לניהול התקשורת - אל תורידו ת'רדים מהמערכת)

ניתן לענות על סעיף זה גם אם לא מצאתם את כל הבעיות בסעיף הקודם.

[10 נקודות]

ג. אחת הסטודנטיות הציעה במקום הפיתרון בסעיף ב, לצמצם את מספר הת'רדים ב-`communicationThreads` לת'רד אחד בלבד. האם הצעתה פותרת את הבעיה? נמקו בקצרה
[5 נקודות]

נספח: קוד הריאקטור, כפי שנלמד בכיתה

```
public class Reactor<T> implements Server<T> {

    private final int port;
    private final Supplier<MessagingProtocol<T>> protocolFactory;
    private final Supplier<MessageEncoderDecoder<T>> readerFactory;
    private final ActorThreadPool<NonBlockingConnectionHandler> pool;
    private Selector selector;

    private Thread selectorThread;
    private final ConcurrentLinkedQueue<Runnable> selectorTasks = new
                                            ConcurrentLinkedQueue<>();

    public Reactor(
            int numThreads,
            int port,
            Supplier<MessagingProtocol<T>> protocolFactory,
            Supplier<MessageEncoderDecoder<T>> readerFactory) {

        this.pool = new ActorThreadPool<>(numThreads);
        this.port = port;
        this.protocolFactory = protocolFactory;
        this.readerFactory = readerFactory;
    }

    public void serve() {
        selectorThread = Thread.currentThread();

        try (   Selector selector = Selector.open();
                ServerSocketChannel serverSock =
                                    ServerSocketChannel.open()) {

            this.selector = selector; //just to be able to close

            serverSock.bind(new InetSocketAddress(port));
            serverSock.configureBlocking(false);
            serverSock.register(selector, SelectionKey.OP_ACCEPT);

            while (!Thread.currentThread().isInterrupted()) {

                selector.select();
                runSelectionThreadTasks();

                for (SelectionKey key : selector.selectedKeys()) {
```

```
                    if (!key.isValid()) {
                        continue;
                    } else if (key.isAcceptable()) {
                        handleAccept(serverSock, selector);
                    } else {
                        handleReadWrite(key);
                    }
                }

                selector.selectedKeys().clear();

            }

        } catch (ClosedSelectorException ex) {
            //do nothing - server was requested to be closed
        } catch (IOException ex) {
            //this is an error
            ex.printStackTrace();
        }

        System.out.println("server closed!!!");
        pool.shutdown();
    }

    void updateInterestedOps(SocketChannel chan, int ops) {
        final SelectionKey key = chan.keyFor(selector);
        if (Thread.currentThread() == selectorThread) {
            key.interestOps(ops);
        } else {
            selectorTasks.add(() -> {
                if(key.isValid())
                    key.interestOps(ops);
            });
            selector.wakeup();
        }
    }



private void handleAccept(ServrSocketChannel serverChan,
                          Selector selector) throws IOException {
        SocketChannel clientChan = serverChan.accept();

        clientChan.configureBlocking(false);
        final NonBlockingConnectionHandler<T> handler = new
```

```
                NonBlockingConnectionHandler<>(
                   readerFactory.get(),
                   protocolFactory.get(),
                   clientChan,
                   this);

         clientChan.register(selector, SelectionKey.OP_READ, handler);
      }

  private void handleReadWrite(SelectionKey key) {
         @SuppressWarnings("unchecked")
         NonBlockingConnectionHandler<T> handler =
                     (NonBlockingConnectionHandler<T>) key.attachment();
         if (key.isReadable()) {
             Runnable task = handler.continueRead();
             if (task != null) {
                 pool.submit(handler, task);
             } else if (!key.isValid()) {
                 return;
             }
         }
         if (key.isWritable()) {
             handler.continueWrite();
         }
      }

   private void runSelectionThreadTasks() {
     while (!selectorTasks.isEmpty()) {
       selectorTasks.remove().run();
     }
   }

   public void close() throws IOException {
     selector.close();
   }
 }

public class NonBlockingConnectionHandler<T> implements
                                            ConnectionHandler<T> {
  private static final int BUFFER_ALLOCATION_SIZE = 1 << 13; //8k
  private static final ConcurrentLinkedQueue<ByteBuffer> BUFFER_POOL =
                                        new ConcurrentLinkedQueue<>();

  private final MessagingProtocol<T> protocol;
  private final MessageEncoderDecoder<T> encdec;
  private final Queue<ByteBuffer> writeQueue = new ConcurrentLinkedQueue<>();
```

```
private final SocketChannel chan;
private final Reactor reactor;

public NonBlockingConnectionHandler(
    MessageEncoderDecoder<T> reader,
    MessagingProtocol<T> protocol,
    SocketChannel chan,
    Reactor reactor) {
      this.chan = chan;
      this.encdec = reader;
      this.protocol = protocol;
      this.reactor = reactor;
}

public Runnable continueRead() {
  ByteBuffer buf = leaseBuffer();
  boolean success = false;
  try {
    success = chan.read(buf) != -1;
  } catch (IOException ex) {
    ex.printStackTrace();
  }
  if (success) {
    buf.flip();
    return () -> {
      try {
        while (buf.hasRemaining()) {
          T nextMessage = encdec.decodeNextByte(buf.get());
          if (nextMessage != null) {
            T response = protocol.process(nextMessage);
            if (response != null) {
              writeQueue.add(ByteBuffer.wrap(
                                encdec.encode(response)));
              reactor.updateInterestedOps(chan, SelectionKey.OP_READ
                                              | SelectionKey.OP_WRITE);
            }
          }
        }
      } finally {
        releaseBuffer(buf);
      }
    };
  } else {
    releaseBuffer(buf);
    close();
    return null;
  }
```

```
    }

  public void close() {
    try {
      chan.close();
    } catch (IOException ex) {
        ex.printStackTrace();
    }
  }

  public void continueWrite() {
    while (!writeQueue.isEmpty()) {
      try {
        ByteBuffer top = writeQueue.peek();
        chan.write(top);
        if (top.hasRemaining()) {
          return;
        } else {
          writeQueue.remove();
        }
      } catch (IOException ex) {
        ex.printStackTrace();
        close();
      }
    }
    if (writeQueue.isEmpty()) {
      if (protocol.shouldTerminate()) close();
        else reactor.updateInterestedOps(chan, SelectionKey.OP_READ);
    }
  }

  private static ByteBuffer leaseBuffer() {
    ByteBuffer buff = BUFFER_POOL.poll();
    if (buff == null) {
      return ByteBuffer.allocateDirect(BUFFER_ALLOCATION_SIZE);
    }
    buff.clear();
    return buff;
  }

  private static void releaseBuffer(ByteBuffer buff) {
    BUFFER_POOL.add(buff);
  }
}
```

```
public class ActorThreadPool<T> {

  private final Map<T, Queue<Runnable>> actors;
  private final ReadWriteLock actorsReadWriteLock;
  private final Set<T> playingNow;
  private final ExecutorService threads;

  public ActorThreadPool(int threads) {
      this.threads = Executors.newFixedThreadPool(threads);
      actors = new WeakHashMap<>();
      playingNow = ConcurrentHashMap.newKeySet();
      actorsReadWriteLock = new ReentrantReadWriteLock();
  }

  public void shutdown() {
    threads.shutdownNow();
  }

  public void submit(T actor, Runnable r) {
    synchronized (actor) {
      if (!playingNow.contains(actor)) {
        playingNow.add(actor);
        execute(r, actor);
      } else {
        pendingRunnablesOf(actor).add(r);
      }
    }
  }

  private Queue<Runnable> pendingRunnablesOf(T actors) {
    actorsReadWriteLock.readLock().lock();
    Queue<Runnable> pendingRunnables = actors.get(actors);
    actorsReadWriteLock.readLock().unlock();

    if (pendingRunnables == null) {
      actorsReadWriteLock.writeLock().lock();
      actors.put(actor, pendingRunnables = new LinkedList<>());
      actorsReadWriteLock.writeLock().unlock();
    }
    return pendingRunnables;
  }

  private void execute(Runnable r, T actor) {
    threads.submit(() -> {
      try {
        r.run();
      } finally {
```

```
          complete(actor);
        }
      });
    }

    private void complete(T actor) {
      synchronized (actor) {
        Queue<Runnable> pending = pendingRunnablesOf(actor);
        if (pending.isEmpty()) {
          playingNow.remove(actor);
        } else {
          execute(pending.poll(), actor);
        }
      }
    }
  }
```

```
public interface MessagingProtocol<T> {
    public T process(T msg);
    public boolean shouldTerminate();
}

public class EchoProtocol implements MessagingProtocol<String> {

    private boolean shouldTerminate = false;

    @Override
    public String process(String msg) {
        shouldTerminate = "bye".equals(msg);
        System.out.println("[" + LocalDateTime.now() + "]: " + msg);
        return createEcho(msg);
    }

    private String createEcho(String message) {
        String echoPart = message.substring(
                Math.max(message.length() - 2, 0), message.length());
        return message + " .. " + echoPart + " .. " + echoPart + " ..";
    }

    @Override
    public boolean shouldTerminate() {
        return shouldTerminate;
    }
}
```

```java
public interface MessageEncoderDecoder<T> {
  T decodeNextByte(byte nextByte);
  byte[] encode(T message);
}

public class LineMessageEncoderDecoder implements MessageEncoderDecoder<String> {

    private byte[] bytes = new byte[1 << 10]; //start with 1k
    private int len = 0;

    @Override
    public String decodeNextByte(byte nextByte) {
         if (nextByte == '\n') {
            return popString();
        }

        pushByte(nextByte);
        return null; //not a line yet
    }

    @Override
    public byte[] encode(String message) {
        return (message + "\n").getBytes(); //uses utf8 by default
    }

    private void pushByte(byte nextByte) {
        if (len >= bytes.length) {
            bytes = Arrays.copyOf(bytes, len * 2);
        }

        bytes[len++] = nextByte;
    }

    private String popString() {
        String result = new String(bytes, 0, len, StandardCharsets.UTF_8);
        len = 0;
        return result;
    }
}
```